

PORTADA

Técnicas de Programación

Unidad 2 — Paradigma Objetual: Del Diagrama de Clases a la Implementación

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

**Dos programadores diseñan el mismo sistema en un tablero.
Uno dibuja una flecha; el otro entiende algo distinto. El
código que escriben después no encaja.**



TRANSICIÓN

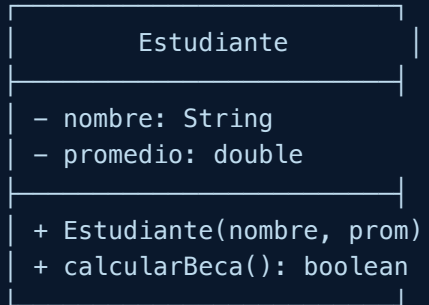
Un lenguaje visual para diseñar antes de programar

Durante toda la Unidad 2 has escrito clases, herencia, interfaces y polimorfismo directamente en código. Pero en un proyecto real, el diseño de las clases y sus relaciones normalmente se dibuja **antes** de escribir una sola línea de Java, usando una notación estándar llamada **UML** (Unified Modeling Language). Hoy aprendes a leer un diagrama de clases UML y a traducirlo directamente a código.

CONCEPTO

La caja de una clase en UML

En un diagrama de clases, cada clase se dibuja como una caja dividida en tres secciones: el **nombre** de la clase, sus **atributos** (con tipo) y sus **métodos** (con parámetros y tipo de retorno).



CONCEPTO

Símbolos de visibilidad

Antes de cada atributo o método, un símbolo indica su nivel de acceso en Java — la misma idea de encapsulamiento que ya usas al declarar atributos `private` con getters y setters `public` .

Símbolo	Visibilidad en Java
+	<code>public</code>
-	<code>private</code>
#	<code>protected</code>



CONCEPTO

Relaciones entre clases: la línea que las conecta

Además de las cajas, un diagrama de clases dibuja **líneas** entre ellas para mostrar cómo se relacionan. La forma de la línea y de sus extremos (flecha, diamante lleno, diamante vacío) cambia el significado — no es un simple adorno.

Sobre la línea de asociación también se escribe la **multiplicidad**: cuántos objetos de un lado se relacionan con cuántos del otro (ej. `1` , `0..1` , `1..*` , `0..*`).

CONTRAEJEMPLO

El error más común: confundir agregación con composición

Ambas se dibujan con un diamante en un extremo de la línea, y ambas representan una relación "tiene un/a" — pero **no** son intercambiables: la diferencia está en si el objeto contenido puede existir de forma independiente al que lo contiene.

CONCEPTO

Composición, agregación y herencia

Relación	Símbolo	Significado
Composición	Diamante lleno	La parte no existe sin el todo (ej. Casa  Habitación : si se destruye la casa, se destruyen sus habitaciones)
Agregación	Diamante vacío	La parte puede existir independiente del todo (ej. Equipo  Jugador : el jugador existe aunque el equipo se disuelva)
Herencia	Flecha triangular vacía	"Es un/a" — la misma relación <code>extends</code> que ya conoces
Implementación	Flecha triangular punteada	"Cumple el contrato de" — la misma relación <code>implements</code> que ya conoces



CASO

Un diagrama completo: sistema de préstamos de biblioteca

Tres clases: Libro (atributos `titulo` , `isbn`), Usuario (atributos `nombre` , `codigo`) y Prestamo , que **compone** la relación entre ambos (un préstamo no existe sin un libro y un usuario asociados). Prestamo tiene multiplicidad 1 hacia Libro y 1 hacia Usuario ; un Usuario puede tener 0..* préstamos activos.

CONCEPTO

Traduciendo el diagrama a código, paso a paso

Cada elemento del diagrama tiene un equivalente directo en Java: los atributos se vuelven campos `private` , los métodos públicos se vuelven métodos `public` , y una asociación con multiplicidad `1` se vuelve una referencia a un objeto (una multiplicidad `0..*` se vuelve una colección, como `ArrayList`).

```
public class Prestamo {  
    private Libro libro;      // multiplicidad 1 hacia Libro  
    private Usuario usuario; // multiplicidad 1 hacia Usuario  
  
    public Prestamo(Libro libro, Usuario usuario) {  
        this.libro = libro;  
        this.usuario = usuario;  
    }  
}
```

CONCEPTO

Cuando la multiplicidad es 0..* : una colección

Si un `Usuario` puede tener varios préstamos activos a la vez, el atributo correspondiente en Java no es un solo objeto, sino una colección de ellos.

```
public class Usuario {  
    private String nombre;  
    private String codigo;  
    private ArrayList<Prestamo> prestamosActivos = new ArrayList<>();  
  
    public void agregarPrestamo(Prestamo p) {  
        prestamosActivos.add(p);  
    }  
}
```

INTERACCIÓN

Del papel al código

En parejas: dibujen en papel un diagrama de clases para un sistema de "pedidos de restaurante" con al menos `Pedido`, `Plato` y `Cliente`, decidiendo qué relación (composición, agregación o asociación simple) y qué multiplicidad usar entre cada par. Luego escriban el esqueleto de las clases en Java a partir de su propio diagrama.

8 minutos · un integrante dibuja, el otro traduce a código

TENSIÓN CONCEPTUAL

El diagrama no dice todo

Un diagrama de clases UML describe **estructura** (qué clases existen, qué atributos y métodos tienen, cómo se relacionan) pero normalmente no describe **comportamiento detallado** (qué hace exactamente el cuerpo de un método, en qué orden se ejecutan los pasos). Para eso existen otros diagramas UML (de secuencia, de actividades) que no cubre este curso — el diagrama de clases es el punto de partida, no el plano completo.

SÍNTESIS

Lo que hay que llevarse de hoy

1. Una clase en UML se dibuja como una caja con nombre, atributos y métodos, cada uno con su símbolo de visibilidad (+ , - , #).
2. Composición (diamante lleno) implica que la parte no existe sin el todo; agregación (diamante vacío) implica que sí puede existir independiente.
3. La herencia y la implementación de interfaces se dibujan con flechas triangulares (vacía sólida y vacía punteada, respectivamente).
4. La multiplicidad de una asociación determina si el atributo correspondiente en Java es un objeto único o una colección.



CIERRE

Antes de la próxima sesión

- Terminar de codificar en el IDE el diagrama de "pedidos de restaurante" diseñado en la interacción de hoy.
- Traer un diagrama de clases propio (de un proyecto personal o inventado) para revisar entre pares la próxima sesión.

La próxima sesión cierra la Unidad 2 con un tema de profundización: lambdas y Streams en Java.

